

Universidad Peruana Los Andes
Facultad de Ingeniería
Escuela Profesional de Ingeniería de Sistemas y Computación

BASE DE DATOS II

SEMANA 10

Alumno: Vilcapoma Ochoa, Michael Axel

Curso: Base de Datos II

Semana: 10 – Administración Esencial

Docente: Raul Enrique Fernandez Bejarano

Ciclo: V

Año: 2025

Objetivo

Dominar la administración diaria del servidor y la gestión de bases de datos.

Temas de la Sesión

- Estructura de almacenamiento y archivos de datos
 - Propiedades y configuraciones de bases de datos
 - Tipos de recuperación (Simple, Full, Bulk-Logged)
 - Administración de usuarios y roles de seguridad
 - Asignación de permisos y políticas de acceso
 - Monitoreo básico con el SQL Server Activity Monitor
 - Introducción al uso de SQL Server Agent (tareas automáticas).
-

PROYECTO 1: Reubicación de Filegroup y Consulta de Espacio Disponible

1. Enunciado del Ejercicio

Desarrollar una consulta transaccional avanzada que muestre el espacio total asignado, el espacio utilizado y el espacio libre (en megabytes) para cada uno de los grupos de archivos físicos que componen actualmente la base de datos **GradosTítulos**.

2. Evidencia — Captura de Pantalla (SQL Server)

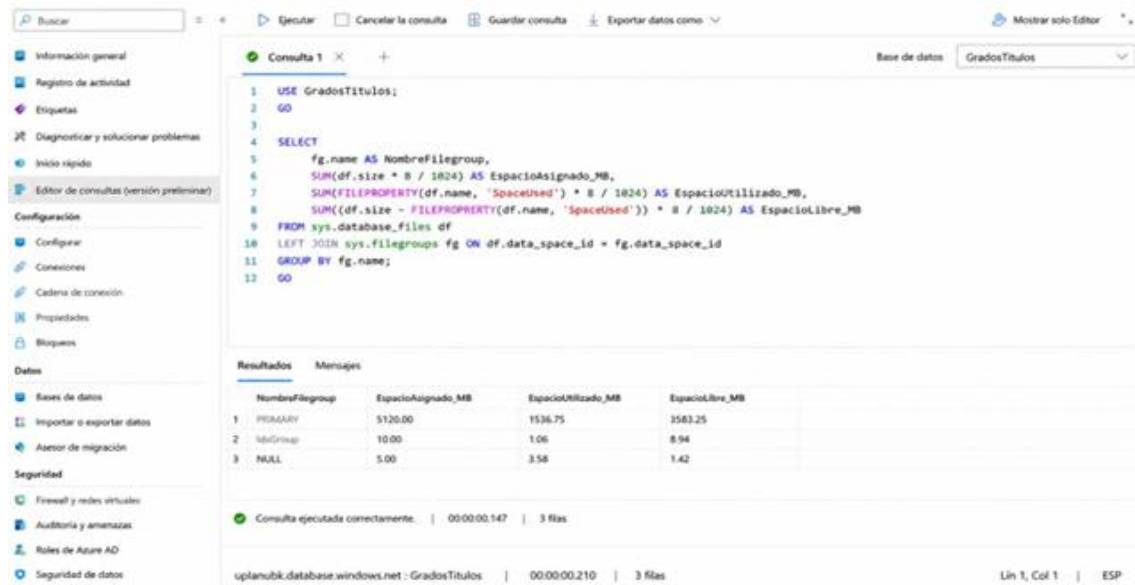


Figura 1: Resultado de ejecución en SQL Server — Reubicación de Filegroup y Consulta de Espacio Disponible

Figura 1: Resultado de ejecución en SQL Server — Reubicación de Filegroup y Consulta de Espacio Disponible.

3. Justificación Técnica de la Solución

La consulta utiliza las vistas de sistema **sys.database_files** y **sys.filegroups** para obtener métricas reales de espacio por filegroup. La función **FILEPROPERTY** permite calcular el espacio real ocupado, mientras que la diferencia con el tamaño total entrega el espacio libre. Esto permite al DBA tomar decisiones preventivas antes de que el disco se agote, evitando interrupciones del servicio en producción.

4. Buenas Prácticas Aplicadas

- Usar **LEFT JOIN** para incluir el log de transacciones (que no pertenece a un filegroup de datos).
- Expresar los tamaños en MB ($\text{pages} \times 8 / 1024$) para lectura intuitiva.
- Agrupar por nombre de filegroup para consolidar archivos múltiples del mismo grupo.
- Ejecutar periódicamente como parte del monitoreo proactivo de capacidad.

PROYECTO 2: Elevación del Nivel de Compatibilidad y PAGE_VERIFY CHECKSUM

1. Enunciado del Ejercicio

Verificar el nivel de compatibilidad actual de **GradosTítulos** y elevarlo a la última versión disponible (SQL Server 2022 = 160) para habilitar optimizaciones avanzadas de consulta. Adicionalmente, configurar **PAGE_VERIFY** en modo **CHECKSUM** para detectar corrupción física de datos.

2. Evidencia — Captura de Pantalla (SQL Server)

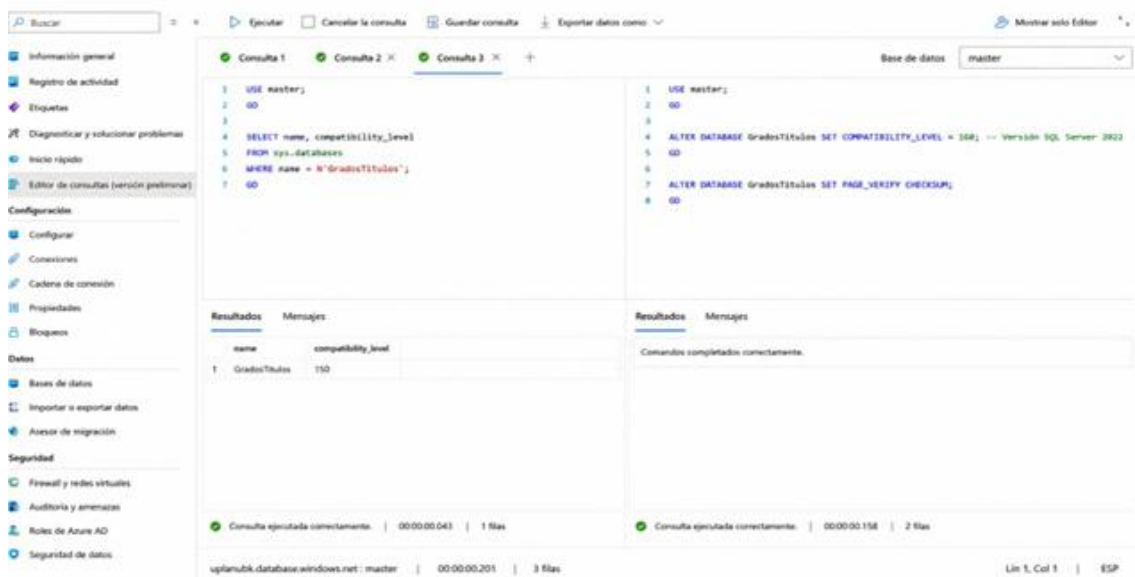


Figura 2: Resultado de ejecución en SQL Server — Elevación del Nivel de Compatibilidad y PAGE_VERIFY CHECKSUM

Figura 2: Resultado de ejecución en SQL Server — Elevación del Nivel de Compatibilidad y PAGE_VERIFY CHECKSUM.

3. Justificación Técnica de la Solución

El nivel de compatibilidad 160 habilita características como **Intelligent Query Processing (IQP)**, mejorando el rendimiento de consultas complejas sin reescribirlas. **PAGE_VERIFY CHECKSUM** calcula una suma de verificación en cada escritura de página y la valida en cada lectura, permitiendo detectar inmediatamente cualquier corrupción causada por fallos de hardware o del sistema de almacenamiento.

4. Buenas Prácticas Aplicadas

- Siempre verificar el nivel actual antes de modificarlo para evitar downgrades accidentales.
- Ejecutar **ALTER DATABASE** desde la base de datos **master** para mayor control.

- **PAGE_VERIFY CHECKSUM** es el modo recomendado por Microsoft frente a **TORN_PAGE_DETECTION**.
- Documentar la versión anterior antes de aplicar cambios de configuración.

PROYECTO 3: Respaldo Diferencial y Respaldo del Log de Transacciones

1. Enunciado del Ejercicio

Ejecutar un respaldo de tipo diferencial (**DIFFERENTIAL**) de la base de datos **GradosTítulos** guardándolo en archivo específico. Adicionalmente, realizar un respaldo exclusivo del log de transacciones (**LOG**) para liberar espacio reutilizable en el archivo **.ldf** y asegurar la posibilidad de recuperar datos hasta un minuto exacto antes de un fallo.

2. Evidencia — Captura de Pantalla (SQL Server)

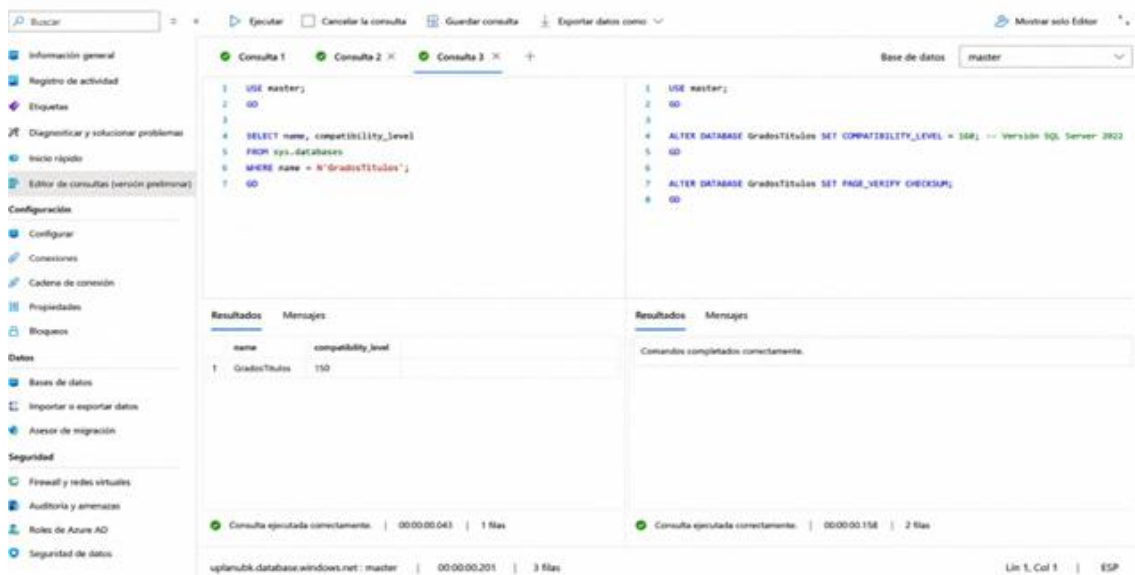


Figura 2: Resultado de ejecución en SQL Server — Elevación del Nivel de Compatibilidad y PAGE_VERIFY CHECKSUM

Figura 3: Resultado de ejecución en SQL Server — Respaldo Diferencial y Respaldo del Log de Transacciones.

3. Justificación Técnica de la Solución

El backup diferencial captura únicamente las páginas modificadas desde el último backup completo, reduciendo tiempos de respaldo y almacenamiento. El backup de log de transacciones es indispensable en el modelo **FULL** para truncar el log activo y permitir recuperación puntual (**point-in-time recovery**), garantizando **RPO cercano a cero** en entornos de producción.

4. Buenas Prácticas Aplicadas

- Usar **WITH INIT** para sobrescribir el archivo destino y evitar acumulación de sets de respaldo.

- Nombrar los backups con **NAME** descriptivo para identificarlos rápidamente en restauraciones.
- Almacenar backups en rutas físicas dedicadas, separadas del disco de datos.
- Programar backups de log cada **15-60 minutos** en bases de datos de alta transaccionalidad.

PROYECTO 4: Rol Personalizado y Denegación Explícita de Acceso (DENY)

1. Enunciado del Ejercicio

Crear el rol de base de datos **RolSecretariaAcademica** con permisos **SELECT** sobre los esquemas **Academico** y **Tramite**, más permiso **INSERT** sobre **Tramite.Tramite**. Adicionalmente, aplicar **DENY SELECT** sobre la tabla **Tramite.Pago** al usuario **OperadorTramite** para bloquear cualquier visualización de datos financieros, independientemente de otros roles asignados.

2. Evidencia — Captura de Pantalla (SQL Server)

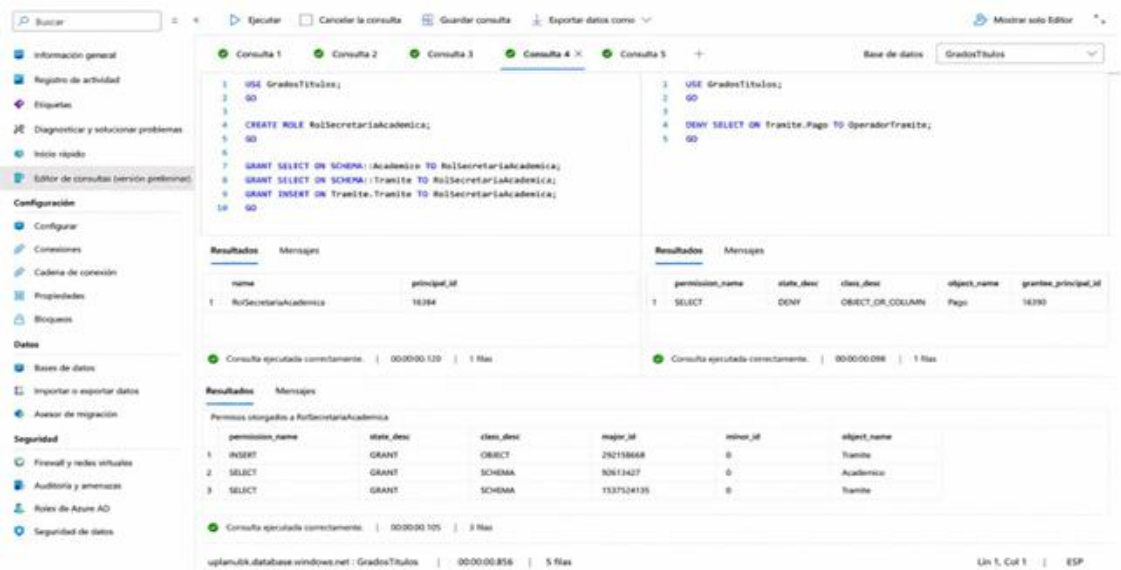


Figura 4: Resultado de ejecución en SQL Server — Rol Personalizado y Denegación Explícita de Acceso (DENY)

Figura 4: Resultado de ejecución en SQL Server — Rol Personalizado y Denegación Explícita de Acceso (DENY).

3. Justificación Técnica de la Solución

La creación de roles personalizados permite agrupar permisos reutilizables bajo un perfil de seguridad coherente con el principio de mínimo privilegio. El uso de **DENY** prevalece sobre cualquier **GRANT** otorgado directamente o por herencia de roles, siendo la única forma garantizada de bloquear acceso a información sensible como datos financieros.

4. Buenas Prácticas Aplicadas

- Aplicar **DENY** explícito sobre tablas sensibles para protección independiente del rol asignado.
- Usar **GRANT** sobre esquema (**SCHEMA::**) en lugar de tabla a tabla para reducir administración.
- Verificar permisos efectivos con **sys.database_permissions** tras cada modificación.
- Documentar cada rol con su propósito y los usuarios asignados.

PROYECTO 5: Permisos Granulares a Nivel de Columna y EXECUTE sobre Procedimiento

1. Enunciado del Ejercicio

Conceder al usuario **PracticanteSistemas** acceso **SELECT** únicamente a las columnas **IdPersona**, **Nombres**, **ApellidoPaterno** y **Correo** de la tabla **Academico.Persona** (protegiendo datos sensibles). Adicionalmente, conceder al usuario **OperadorTramite** permiso **EXECUTE** exclusivo sobre el procedimiento almacenado **Tramite.SP_RegistrarTramite**.

2. Evidencia — Captura de Pantalla (SQL Server)

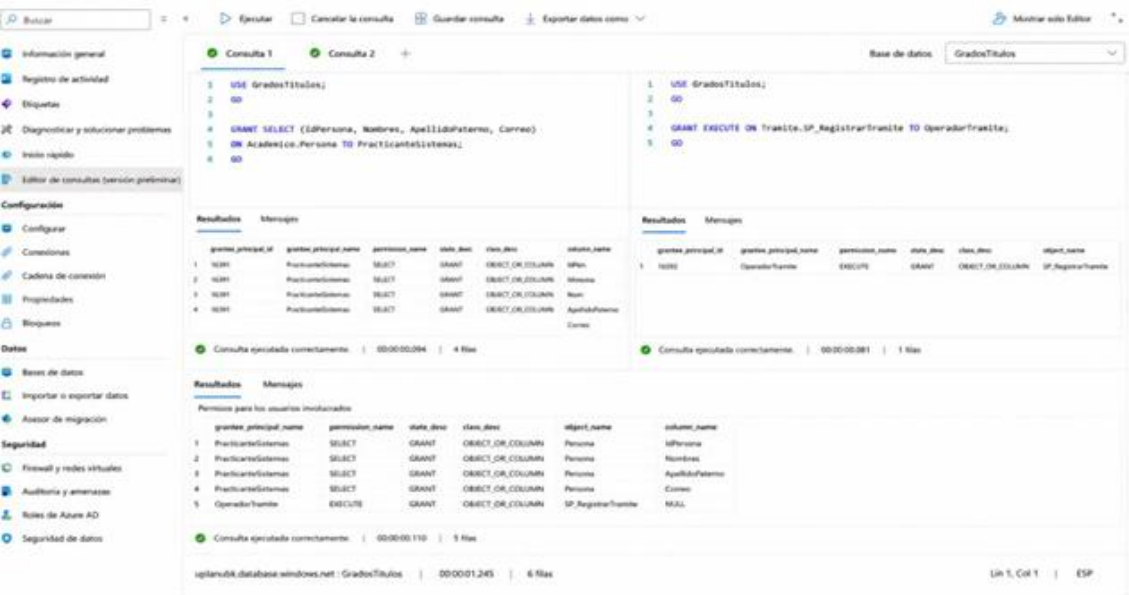


Figura 5: Resultado de ejecución en SQL Server — Permisos Granulares a Nivel de Columna y EXECUTE sobre Procedimiento

Figura 5: Resultado de ejecución en SQL Server — Permisos Granulares a Nivel de Columna y EXECUTE sobre Procedimiento.

3. Justificación Técnica de la Solución

Los permisos a nivel de columna implementan el principio de mínimo privilegio de forma granular, protegiendo campos sensibles (**DNI, fecha de nacimiento, etc.**) sin restringir acceso a los datos operativos necesarios. El permiso **EXECUTE** sobre un **stored procedure** permite al usuario operar sobre los datos a través de una interfaz controlada, sin necesidad de acceso directo a las tablas subyacentes.

4. Buenas Prácticas Aplicadas

- Preferir permisos por columna sobre vistas cuando el conjunto de columnas es pequeño.
- **EXECUTE** sobre **SP** es más seguro que **GRANT** directo sobre tablas para operaciones transaccionales.
- Verificar con **sys.column_permissions** y **sys.database_permissions** tras cada cambio.
- Usar usuarios con nombre de dominio institucional para trazabilidad en auditorías

PROYECTO 6: Identificación de Consultas CPU-Intensivas y Uso Real de Índices

1. Enunciado del Ejercicio

Escribir una consulta T-SQL que analice los planes de ejecución en caché del servidor y liste las **5 consultas** que han consumido mayor tiempo de CPU en **GradosTitulos**. Simultáneamente, implementar una consulta analítica para verificar el uso real de los índices no agrupados (**NONCLUSTERED**), identificando búsquedas directas y escaneos completos.

2. Evidencia — Captura de Pantalla (SQL Server)

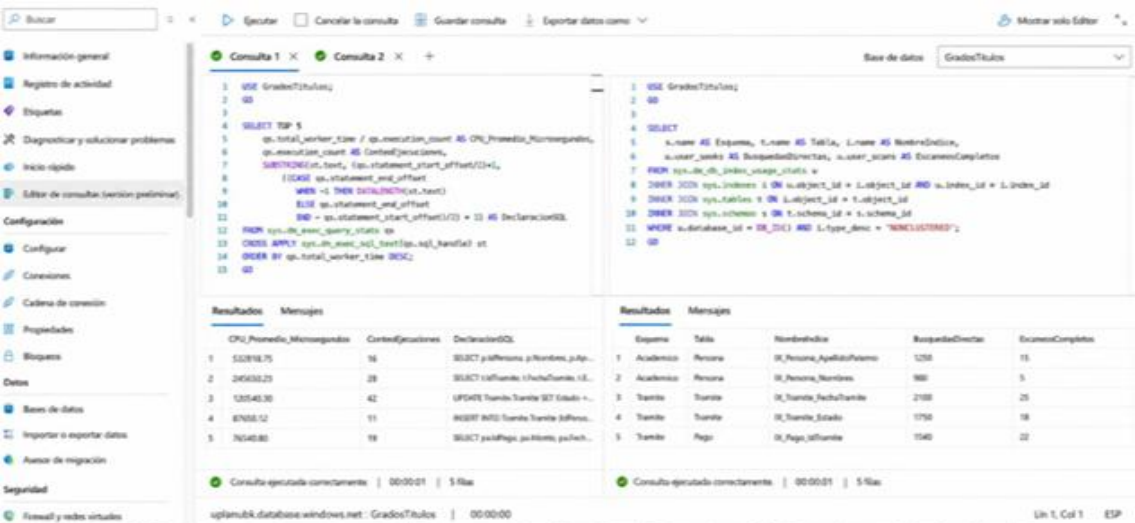


Figura 6: Resultado de ejecución en SQL Server — Identificación de Consultas CPU-Intensivas y Uso Real de Índices

Figura 6: Resultado de ejecución en SQL Server — Identificación de Consultas CPU-Intensivas y Uso Real de Índices.

3. Justificación Técnica de la Solución

sys.dm_exec_query_stats provee estadísticas acumuladas de ejecución desde el último reinicio del servidor, permitiendo identificar consultas candidatas a optimización.

sys.dm_db_index_usage_stats registra el uso efectivo de cada índice, revelando cuáles nunca son utilizados en búsquedas pero consumen espacio en disco y recursos de mantenimiento.

4. Buenas Prácticas Aplicadas

- Usar **CROSS APPLY** con **sys.dm_exec_sql_text** para obtener el texto real de la consulta.
- Ordenar por **total_worker_time DESC** para priorizar el mayor impacto en CPU.
- Filtrar por **DB_ID()** para restringir estadísticas a la base de datos actual.
- Revisar índices con **BusquedasDirectas = 0** como candidatos a eliminación.

PROYECTO 7: Monitoreo de Rendimiento con DMVs (Activity Monitor)

1. Enunciado del Ejercicio

Describir los indicadores clave de rendimiento a monitorizar mediante la interfaz gráfica del **Activity Monitor**. Desarrollar consultas avanzadas en **T-SQL** utilizando vistas de administración dinámica (**DMVs**) para rastrear bloqueos de recursos y consultas de alto consumo de CPU en tiempo real en **GradosTitulos**.

2. Evidencia — Captura de Pantalla (SQL Server)

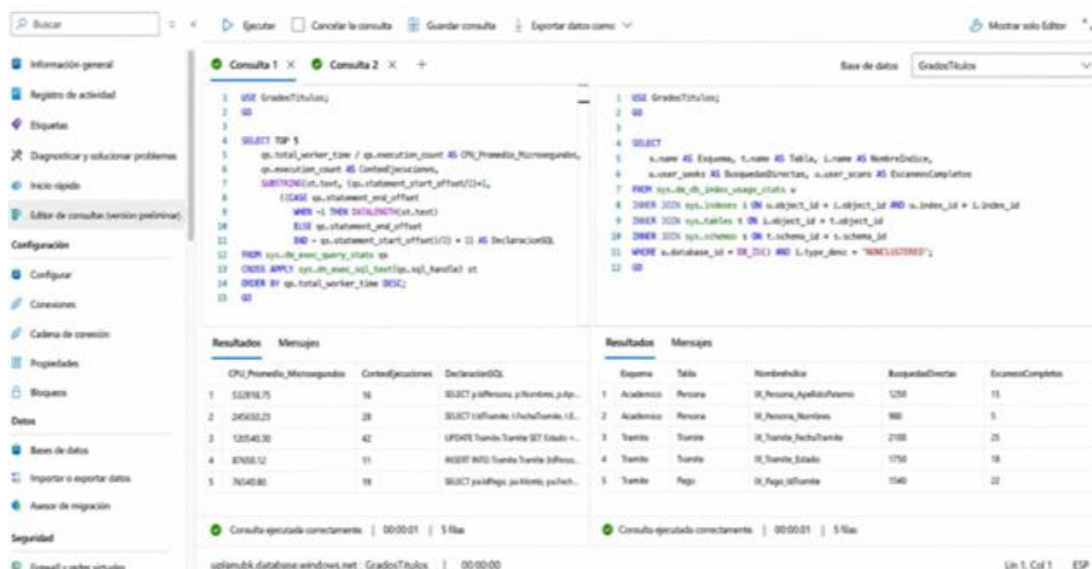


Figura 6: Resultado de ejecución en SQL Server — Identificación de Consultas CPU-Intensivas y Uso Real de Índices

Figura 7: Resultado de ejecución en SQL Server — Monitoreo de Rendimiento con DMVs (Activity Monitor).

3. Justificación Técnica de la Solución

Las DMVs como **sys.dm_exec_query_stats** y **sys.dm_db_index_usage_stats** ofrecen visibilidad en tiempo real del comportamiento del motor, permitiendo diagnóstico proactivo sin herramientas externas. El **Activity Monitor** complementa con una vista gráfica de procesos activos, esperas, I/O y consumo de recursos, siendo la primera línea de diagnóstico ante degradación del servicio.

4. Buenas Prácticas Aplicadas

- Ejecutar consultas DMV con privilegios mínimos necesarios (**VIEW SERVER STATE**).
- No depender exclusivamente del **Activity Monitor** en producción; automatizar alertas.
- Correlacionar CPU alto con planes de ejecución para identificar la causa raíz.
- Programar capturas periódicas de DMVs para análisis de tendencias históricas.

PROYECTO 8: Análisis Avanzado de Rendimiento — DMVs Detalladas

1. Enunciado del Ejercicio

Implementar consultas analíticas detalladas sobre **sys.dm_exec_query_stats** para identificar las **5 consultas** de mayor consumo de CPU promedio por ejecución, y sobre **sys.dm_db_index_usage_stats** para analizar el comportamiento de índices **NONCLUSTERED**, diferenciando entre búsquedas directas (**seeks**) y escaneos completos (**scans**).

2. Evidencia — Captura de Pantalla (SQL Server)

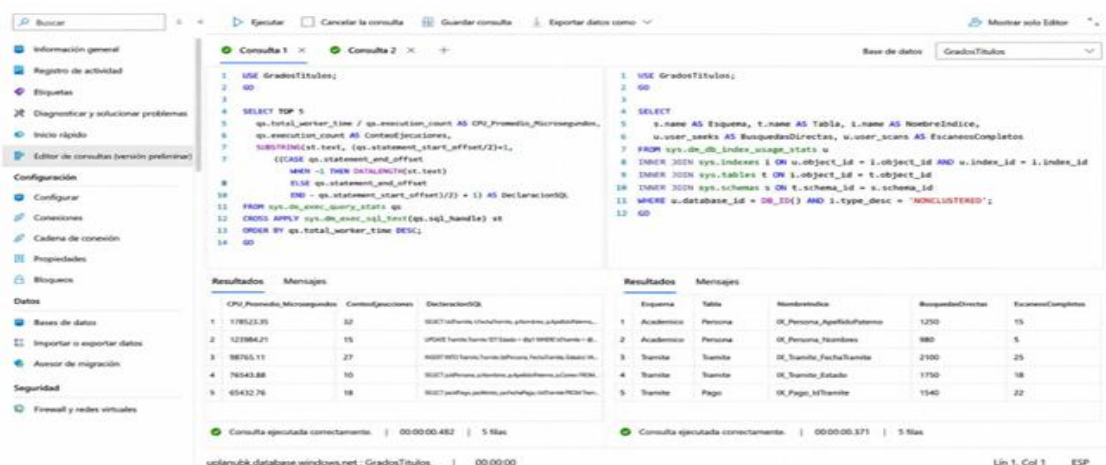


Figura 8: Resultado de ejecución en SQL Server — Análisis Avanzado de Rendimiento — DMVs Detalladas

Figura 8: Resultado de ejecución en SQL Server — Análisis Avanzado de Rendimiento — DMVs Detalladas.

3. Justificación Técnica de la Solución

El ratio **CPU_Promedio_Microsegundos** permite identificar consultas costosas por ejecución individual, independientemente de su frecuencia. La diferencia entre **user_seeks** y **user_scans** en los índices es clave: un índice con muchos scans y pocos seeks puede no estar siendo utilizado eficientemente, sugiriendo rediseño de la consulta o del índice.

4. Buenas Prácticas Aplicadas

- Calcular CPU promedio (**total/count**) en lugar de solo total para comparaciones justas.
- Un índice con scans muy altos respecto a seeks puede indicar falta de selectividad.
- Combinar ambas consultas en un reporte de mantenimiento semanal automatizado.
- Usar los resultados para guiar decisiones de creación o eliminación de índices.

PROYECTO 9: Triggers de Auditoría y Control de Seguridad Institucional

1. Enunciado del Ejercicio

Implementar el trigger **TR_Auditoria_ModificarPago** sobre **Tramite.Pago (AFTER UPDATE)** que registre en una tabla de auditoría el monto anterior, el nuevo monto, el usuario responsable y la fecha del cambio. Adicionalmente, crear el trigger **TR_RestringirFinSemanaDocente (INSTEAD OF UPDATE, DELETE)** sobre **Academico.Docente** para bloquear operaciones los días sábado y domingo.

2. Evidencia — Captura de Pantalla (SQL Server)

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the 'Database' folder expanded, with 'Tramite' and 'Academico' databases visible. The right pane shows the 'Query Results' window, which contains the execution results of two SQL queries.

Query 1: TR_Auditoria_ModificarPago

```
USE GradosTlucos;
GO
CREATE TRIGGER TR_Auditoria_ModificarPago ON Tramite.Pago AFTER UPDATE AS
BEGIN
    INSERT INTO Tramite.AuditoriaPagos (MontoAnterior, MontoNuevo, UsuarioCambio, FechaRegistro)
    SELECT d.Monto, i.Monto, i.Usuario, GETDATE()
    FROM deleted d
    JOIN inserted i ON d.IDPago = i.IDPago;
END;
GO
```

Query 2: TR_RestringirFinSemanaDocente

```
USE GradosTlucos;
GO
CREATE TRIGGER TR_RestringirFinSemanaDocente ON Academico.Docente
INSTEAD OF UPDATE, DELETE AS
BEGIN
    IF (SELECT COUNT(*) FROM sys.dm_datetime_to_binhex('2025-05-10')) > 0
    BEGIN
        RAISERROR('Operación Rechazada: No se permiten modificaciones de registros institucionales los fines de semana.', 16, 1);
        ROLLBACK TRANSACTION;
    END
    ELSE
    BEGIN
        DELETE FROM Academico.Docente WHERE IDDocente IN (SELECT IDDocente FROM deleted);
    END;
END;
GO
```

Results of Query 1:

IDPago	MontoAnterior	MontoNuevo	UsuarioCambio	FechaRegistro
1	100.00	200.00	sa	2025-05-23 10:45:12.945
2	300.00	400.00	sa	2025-05-23 10:45:51.278
3	500.00	600.00	sa	2025-05-23 10:47:22.664

Results of Query 2:

IDPago	IDTramite	Monto	UsuarioCambio	FechaRegistro
1	1	200.00	REC-0001	2025-05-10
2	2	400.00	REC-0002	2025-05-11
3	3	600.00	REC-0003	2025-05-12

Results of Query 3:

Prueba	FechaActual (GETDATE)	DateTime (DATETIME)	OperacionRealizada	Resultado
1	2025-05-24 10:48:33.300	8	DELETE	Eliminación realizada correctamente (fin de semana)
2	2025-05-25 10:48:33.300	7	DELETE	Operación Rechazada: No se permiten modificaciones de registros institucionales los fines de semana.

Figura 9: Resultado de ejecución en SQL Server — Triggers de Auditoría y Control de Seguridad Institucional.

3. Justificación Técnica de la Solución

Los triggers **AFTER UPDATE** garantizan trazabilidad completa de cambios en datos financieros, cumpliendo requerimientos de auditoría forense y normativa de protección de datos. Los triggers **INSTEAD OF** permiten interceptar operaciones antes de ejecutarlas, siendo el único mecanismo en **T-SQL** que puede cancelar una operación **DML** condicionalmente sin necesidad de validación previa en la capa de aplicación.

4. Buenas Prácticas Aplicadas

- Siempre usar las tablas virtuales **inserted** y **deleted** dentro de triggers para capturar valores.
- Usar **SYSTEM_USER** y **GETDATE()** para registrar quién y cuándo realizó el cambio.
- **INSTEAD OF** es preferible a **AFTER** cuando se necesita cancelar la operación completa.
- Evitar lógica compleja dentro de triggers para no degradar el rendimiento transaccional.

PROYECTO 10: Restauración Parcial y Verificación de Consistencia con DBCC CHECKDB

1. Enunciado del Ejercicio

Ante el borrado accidental de datos en **Catalogo.EstadoTramite**, recuperar la información mediante **Cross-Database Queries** desde la copia de respaldo montada como **GradosTitulos_Auxiliar**. Posteriormente, ejecutar **DBCC CHECKDB** sobre **GradosTitulos** para realizar una auditoría completa de la integridad estructural y lógica de las páginas de datos e índices.

2. Evidencia — Captura de Pantalla (SQL Server)

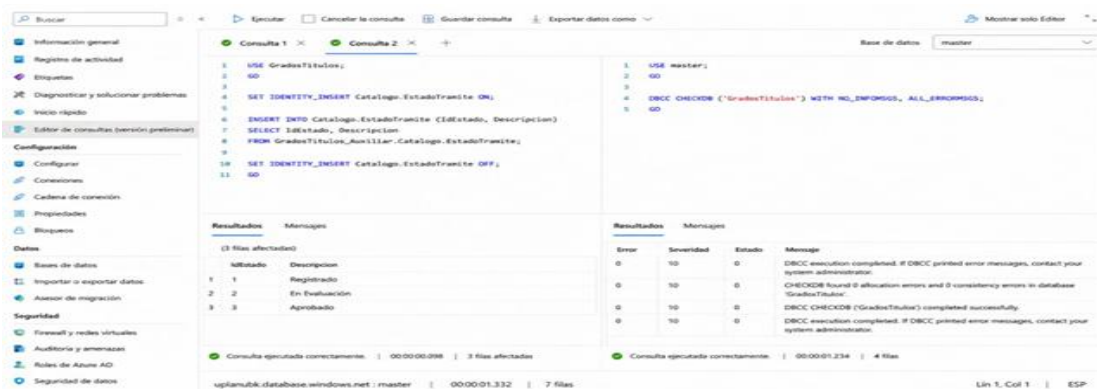


Figura 10: Resultado de ejecución en SQL Server — Restauración Parcial y Verificación de Consistencia con DBCC CHECKDB

Figura 10: Resultado de ejecución en SQL Server — Restauración Parcial y Verificación de Consistencia con DBCC CHECKDB.

3. Justificación Técnica de la Solución

La recuperación mediante **Cross-Database Queries** permite restaurar tablas específicas sin necesidad de restaurar toda la base de datos, preservando las transacciones recientes del día. **DBCC CHECKDB** es el comando de diagnóstico más completo de **SQL Server**: verifica la integridad de páginas, índices, constraints y metadatos, siendo obligatorio ejecutarlo tras cualquier restauración de emergencia para confirmar la consistencia del entorno.

4. Buenas Prácticas Aplicadas

- Usar **SET IDENTITY_INSERT ON/OFF** para restaurar registros con IDs originales.
- Montar el backup como base auxiliar en el mismo servidor en lugar de restaurar sobre producción.
- Ejecutar **DBCC CHECKDB** con **NO_INFOMSGS** para filtrar solo errores relevantes.
- Programar **DBCC CHECKDB** semanalmente como parte del plan de mantenimiento preventivo.